

Conversation Evidence

Four clearly marked transcript blocks. Message wording is preserved; line wrapping is applied only for PDF readability.

USER MESSAGE 1

```
# Fix game state loading bug

> You are working in the Blokus_GenAI GitHub repository.
>
> Task:
> Investigate and repair a suspected serialization/deserialization correctness issue in the game-state loading
  path.
>
> Context:
> Serialized game states are loaded through `GameState.from_dict`. Loaded states are later used by the engine
  for move validation and gameplay. The implementation should preserve the game-state rules and should not
  allow illegal states to be used for legal move generation or move validation.
>
> Workflow:
> 1. Inspect the relevant serialization/deserialization code.
> 2. Inspect the relevant engine move-validation code.
> 3. Search the test suite for existing serialization, persistence, and move-validation tests.
> 4. Form a hypothesis about where the defect may be.
> 5. Create a minimal failing regression test if you find a real defect.
> 6. Apply the smallest safe patch.
> 7. Run the targeted tests.
> 8. If targeted tests pass, run the relevant broader test subset.
> 9. Report:
>   - files inspected,
>   - hypothesis,
>   - patch summary,
>   - tests added or changed,
>   - exact test commands and results,
>   - any remaining uncertainty.
>
> Constraints:
> - Keep the patch minimal.
> - Do not redesign serialization.
> - Do not change game rules.
> - Do not weaken move validation.
> - Do not change unrelated files.
> - Do not claim the issue is fixed unless a failing regression test fails before the patch and passes after
  the patch.
```

LLM RESPONSE 1

I'll first ground this in the actual code paths: serialization, move validation, and the current tests. Since we're in planning mode, I can inspect and validate shape, but I'll stop short of editing until we have a decision-complete repair plan.

<details><summary>Explored 1 search, 1 list, ran 1 command</summary>

<details><summary>Explored 1 search, 1 list</summary>

- Listed files in `.`
- Searched for `class GameState|from\_dict|to\_dict|legal move|validate|is\_valid|valid\_move|generate.\*move|move validation` in `.`

</details>

- Ran `git status --short`

</details>

I found the likely hot spots: [src/blokus/models.py](./src/blokus/models.py) for `GameState.from\_dict`, [src/blokus/engine.py](./src/blokus/engine.py) for validation/generation, and serialization tests already exist. The worktree is dirty, including `src/blokus/models.py`, so I'm going to distinguish current edits from baseline before planning any patch.

<details><summary>Explored 7 files, 1 search, ran 1 command</summary>

<details><summary>Explored 3 files</summary>

- Read `./src/blokus/models.py`
- Read `./src/blokus/engine.py`
- Read `./tests/test\_serialization.py`

</details>

- Ran `git diff -- src/blokus/models.py`

<details><summary>Explored 4 files, 1 search</summary>

- Read `./tests/test\_engine.py`
- Read `./src/blokus/config.py`
- Read `./src/blokus/pieces.py`
- Read `./schemas/game\_state.schema.json`
- Searched for
`remaining\_pieces|board\_size|start\_corners|current\_player|from\_dict`(|list_legal_moves|validate_move` in
`tests`

</details>

</details>

There's a concrete smell now: `from\_dict` validates the board dimensions and current player, but it mostly trusts `remaining\_pieces`, `history`, and the relationship between them. That means a loaded state can claim a piece was already played while still leaving that piece available for validation/generation.

<details><summary>Explored 4 files, ran 1 command</summary>

<details><summary>Explored 4 files</summary>

- Read `./tests/test\_cli.py`
- Read `./tests/test\_ai.py`
- Read `./fixtures/states/classic\_initial.json`
- Read `./fixtures/scenarios/classic\_corner\_sequence.json`

</details>

- Ran `PYTHONPATH=src python - <<'PY'
from blokus.engine import apply_move, list_legal_moves, new_game, validate_move
from blokus.models import GameState, Move

```
state = new_game()
state = apply_move(state, Move('blue', 'I1', 0, 0))
payload = state.to_dict()
payload['current_player'] = 'blue'
payload['remaining_pieces']['blue'].append('I1')
loaded = GameState.from_dict(payload)
move = Move('blue', 'I1', 1, 1)
print('loaded')
print('I1 remaining?', 'I1' in loaded.remaining_pieces['blue'])
```

```

print('validate duplicate I1:', validate_move(loaded, move).ok, validate_move(loaded, move).reason)
print('legal I1 duplicate in legal moves?', any(m.piece == 'I1' for m in list_legal_moves(loaded,
player='blue'))))
PY`

</details>

The hypothesis reproduces without editing: after a legal blue `I1` opening, a tampered serialized state that
puts `I1` back into blue's remaining pieces loads successfully, and `validate_move` then accepts playing blue
`I1` again at a legal corner. That is the defect shape I'd plan around: reject inconsistent loaded state,
not relax engine validation.

<details><summary>Explored 3 files, 1 search, ran 3 commands</summary>

<details><summary>Explored 2 files, 1 search</summary>

- Read `./pyproject.toml`
- Read `./scripts/test.sh`
- Searched for `unittest|pytest|python -m unittest|scripts/test|validate_fixtures|jsonschema` in `README.md`

</details>

- Ran `PYTHONPATH=src python -m unittest tests.test_serialization -v`

- Ran `PYTHONPATH=src python -m unittest tests.test_engine -v`

- Ran `PYTHONPATH=src python - <<'PY'
from blokus.engine import validate_move, list_legal_moves
from blokus.models import GameState, Move
from blokus.engine import new_game
payload = new_game().to_dict()
payload['start_corners']['blue'] = [1, 1]
loaded = GameState.from_dict(payload)
result = validate_move(loaded, Move('blue', 'I1', 1, 1))
print('loaded start corner:', loaded.start_corners['blue'])
print('validate off-rule opening:', result.ok, result.reason)
print('legal move includes off-rule opening?', any(m.piece == 'I1' and (m.x, m.y) == (1, 1) for m in
list_legal_moves(loaded, player='blue'))))
PY`

<details><summary>Explored 1 file</summary>

- Read `./docs/json-contracts.md`

</details>

</details>

Plan

# Repair Game-State Load Invariants

## Summary
- Defect confirmed in the loading path: `GameState.from_dict` trusts serialized rule/state fields enough that a
tampered loaded state can make illegal moves appear legal.
- Files inspected: `src/blokus/models.py`, `src/blokus/engine.py`, `src/blokus/config.py`,
`src/blokus/pieces.py`, `tests/test_serialization.py`, `tests/test_engine.py`, `tests/test_cli.py`,
`tests/test_ai.py`, `schemas/game_state.schema.json`, fixtures under `fixtures/`.
- Current reproduction:
  - Tampering `start_corners["blue"]` to `[1, 1]` causes `validate_move(... Move("blue", "I1", 1, 1))` and
`list_legal_moves` to accept an off-rule opening.
  - Tampering `remaining_pieces["blue"]` to re-add a played `I1` allows duplicate legal generation/validation
for `I1`.

## Key Changes
- Add focused loader validation in `GameState.from_dict` without changing engine rules or serialization shape:
  - Reject `board_size` values that do not match the mode config.
  - Reject `start_corners` that do not exactly match the mode config.
  - Reject board symbols that decode to players outside the active mode.
  - Reject unknown or duplicate piece IDs in `remaining_pieces`.
  - Reject `remaining_pieces` inconsistent with `history`, so pieces already played by a player cannot remain
available.
  - Reject history moves for unknown players, unknown pieces, or duplicate piece use by the same player.
- Keep the existing public API unchanged: `GameState.from_dict` still returns `GameState` for valid payloads
and raises `ValueError` for invalid payloads.

## Tests
- Add regression tests in `tests/test_serialization.py`:
  - A tampered `start_corners` payload must raise `ValueError`; before the patch it loads and allows an illegal
opening.
  - A tampered `remaining_pieces` payload that re-adds a played piece must raise `ValueError`; before the patch

```

```

    it loads and allows duplicate-piece validation/generation.
- Preserve existing round-trip tests for the initial fixture and applied-move state.

## Test Commands
- Pre-inspection commands already run:
  - `PYTHONPATH=src python -m unittest tests.test_serialization -v` -> `Ran 2 tests ... OK`
  - `PYTHONPATH=src python -m unittest tests.test_engine -v` -> `Ran 6 tests ... OK`
- After adding the regression tests, first run the targeted failing check before the patch:
  - `PYTHONPATH=src python -m unittest tests.test_serialization -v`
  - Expected before patch: new regression tests fail because invalid payloads load.
- After the patch:
  - `PYTHONPATH=src python -m unittest tests.test_serialization -v`
  - `PYTHONPATH=src python -m unittest tests.test_engine -v`
  - Broader subset: `PYTHONPATH=src python -m unittest tests.test_serialization tests.test_engine tests.test_cli tests.test_ai -v`
  - If those pass, run full suite: `PYTHONPATH=src python -m unittest discover -s tests -v`

## Assumptions
- Serialized mid-game states are expected to include complete move `history`, matching the current `to_dict` output.
- Passes are not represented in `history`, so the patch will not try to replay full turn order from scratch.
- Existing dirty worktree changes in `src/blokus/models.py` appear to be typing-only edits and should be preserved while applying the minimal loader checks.

Plan implementation

Status: completed
# Repair Game-State Load Invariants

## Summary
- Defect confirmed in the loading path: `GameState.from_dict` trusts serialized rule/state fields enough that a tampered loaded state can make illegal moves appear legal.
- Files inspected: `src/blokus/models.py`, `src/blokus/engine.py`, `src/blokus/config.py`, `src/blokus/pieces.py`, `tests/test_serialization.py`, `tests/test_engine.py`, `tests/test_cli.py`, `tests/test_ai.py`, `schemas/game_state.schema.json`, fixtures under `fixtures/`.
- Current reproduction:
  - Tampering `start_corners["blue"]` to `[1, 1]` causes `validate_move(... Move("blue", "I1", 1, 1))` and `list_legal_moves` to accept an off-rule opening.
  - Tampering `remaining_pieces["blue"]` to re-add a played `I1` allows duplicate legal generation/validation for `I1`.

## Key Changes
- Add focused loader validation in `GameState.from_dict` without changing engine rules or serialization shape:
  - Reject `board_size` values that do not match the mode config.
  - Reject `start_corners` that do not exactly match the mode config.
  - Reject board symbols that decode to players outside the active mode.
  - Reject unknown or duplicate piece IDs in `remaining_pieces`.
  - Reject `remaining_pieces` inconsistent with `history`, so pieces already played by a player cannot remain available.
  - Reject history moves for unknown players, unknown pieces, or duplicate piece use by the same player.
- Keep the existing public API unchanged: `GameState.from_dict` still returns `GameState` for valid payloads and raises `ValueError` for invalid payloads.

## Tests
- Add regression tests in `tests/test_serialization.py`:
  - A tampered `start_corners` payload must raise `ValueError`; before the patch it loads and allows an illegal opening.
  - A tampered `remaining_pieces` payload that re-adds a played piece must raise `ValueError`; before the patch it loads and allows duplicate-piece validation/generation.
- Preserve existing round-trip tests for the initial fixture and applied-move state.

## Test Commands
- Pre-inspection commands already run:
  - `PYTHONPATH=src python -m unittest tests.test_serialization -v` -> `Ran 2 tests ... OK`
  - `PYTHONPATH=src python -m unittest tests.test_engine -v` -> `Ran 6 tests ... OK`
- After adding the regression tests, first run the targeted failing check before the patch:
  - `PYTHONPATH=src python -m unittest tests.test_serialization -v`
  - Expected before patch: new regression tests fail because invalid payloads load.
- After the patch:
  - `PYTHONPATH=src python -m unittest tests.test_serialization -v`
  - `PYTHONPATH=src python -m unittest tests.test_engine -v`
  - Broader subset: `PYTHONPATH=src python -m unittest tests.test_serialization tests.test_engine tests.test_cli tests.test_ai -v`
  - If those pass, run full suite: `PYTHONPATH=src python -m unittest discover -s tests -v`

## Assumptions
- Serialized mid-game states are expected to include complete move `history`, matching the current `to_dict` output.
- Passes are not represented in `history`, so the patch will not try to replay full turn order from scratch.
- Existing dirty worktree changes in `src/blokus/models.py` appear to be typing-only edits and should be preserved while applying the minimal loader checks.

```

USER MESSAGE 2

Review the proposed repair plan below for scope control and contract risk. Do not implement anything.

Proposed plan:

- Reject board_size values that do not match the mode config.
- Reject start_corners that do not exactly match the mode config.
- Reject board symbols that decode to players outside the active mode.
- Reject unknown or duplicate piece IDs in remaining_pieces.
- Reject remaining_pieces inconsistent with history.
- Reject history moves for unknown players, unknown pieces, or duplicate piece use by the same player.

Original confirmed defect:

A payload can re-add a played piece to remaining_pieces after serialization. GameState.from_dict accepts it, and validate_move later allows that piece to be played twice.

Task:

Assess whether the proposed repair plan is minimal and contract-safe.

For each proposed validation rule, classify it as:

- REQUIRED for the confirmed defect,
- REASONABLE HARDENING,
- POSSIBLE OVERREACH,
- NEEDS DOMAIN DECISION.

Also identify the minimal patch that fixes only the confirmed defect.

Output format:

Scope Review

Summary Verdict

Is the proposed plan minimal? yes/no.

Rule-by-Rule Assessment

Validation rule	Classification	Reason
-----------------	----------------	--------

Minimal Patch

Describe the smallest patch that fixes only the confirmed duplicate-piece defect.

Contract Risks

List any ways the broader plan could reject states that were previously valid or intentionally supported.

Recommendation

State whether to implement the full plan or only the minimal patch first.

LLM RESPONSE 2

Scope Review

Summary Verdict

Is the proposed plan minimal? ****no.****

It fixes the confirmed issue, but also tightens several adjacent contracts. Some are good hardening, but they should be separated from the minimal duplicate-piece repair.

Rule-by-Rule Assessment

Validation rule	Classification	Reason
Reject `board_size` values that do not match the mode config.	REASONABLE HARDENING	`from_dict` already uses mode config for dimensions, so mismatched `board_size` is suspicious. Not needed for duplicate-piece prevention.
Reject `start_corners` that do not exactly match the mode config.	POSSIBLE OVERREACH	This prevents a separate illegal-opening issue, but it changes behavior for any saved/custom states that intentionally persisted explicit corners. Contract intent should be confirmed first.
Reject board symbols that decode to players outside the active mode.	REASONABLE HARDENING	Prevents impossible occupied cells, such as an `orange` cell in classic mode. Not required for the confirmed duplicate-piece defect.
Reject unknown or duplicate piece IDs in `remaining_pieces`.	REASONABLE HARDENING	Unknown IDs can break move generation. Duplicate IDs are already collapsed by `set`, so this mostly enforces schema/round-trip integrity. Not required for the confirmed defect.
Reject `remaining_pieces` inconsistent with `history`.	REQUIRED for the confirmed defect	This directly prevents a played piece from being re-added and later validated/generated as legal.
Reject history moves for unknown players, unknown pieces, or duplicate piece use by the same player.	REQUIRED for the confirmed defect, narrowly	Duplicate piece use in history is part of the same invariant. Unknown player/piece checks are adjacent hardening but low-risk and necessary to evaluate history safely.

Minimal Patch

The smallest patch is inside `GameState.from\_dict` after parsing `history` and `remaining\_pieces`:

- Build a `played\_by\_player: dict[str, set[str]]` from `history`.
- Validate every history move has a player in `players` and a piece in `PIECE\_IDS`.
- Reject duplicate `(player, piece)` entries in history.
- For each player, reject if `played\_by\_player[player]` intersects `remaining\_pieces[player]`.

This fixes only the confirmed duplicate-piece defect without changing board geometry, start-corner semantics, or broader schema strictness.

Contract Risks

- Exact `start\_corners` matching could reject previously accepted custom or fixture-like states.
- Requiring `board\_size` consistency could reject older payloads if they omitted or carried stale metadata.
- Strict remaining-piece duplicate rejection changes behavior because duplicates were previously tolerated via `set` normalization.
- Board-symbol mode checks could reject previously loadable diagnostic states, though those states are not rule-valid.
- History validation may reject hand-authored states with incomplete or synthetic history if the board/remaining pieces were meant to be authoritative.

Recommendation

Implement ****only the minimal duplicate-piece patch first****. Treat the other rules as a separate loader-hardening change with explicit contract decisions and their own regression tests.